

Analisis Perbandingan UCS, A*, dan RRT* pada Simulasi Pencarian Jalur Robot Sepak Bola Humanoid di Lapangan Berhalangan

Muchammad Yuda Tri Ananda - 24060124110142
Departemen Informatika
Universitas Diponegoro
Semarang, Indonesia
myudak@students.undip.ac.id

Abstract—Penelitian ini membahas penerapan Brute Force, kelompok Uniform Cost Search (UCS), Greedy Best First Search (GBFS), A*, dan Rapidly-exploring Random Tree Star (RRT*) pada simulasi pencarian jalur robot sepak bola humanoid di lapangan berhalangan. Sesuai ketentuan proyek, UCS, GBFS, dan A* diperlakukan sebagai satu kelompok strategi pencarian berbasis biaya dan heuristik, sedangkan Brute Force menjadi algoritma daftar kedua dan RRT* menjadi algoritma di luar daftar. Eksperimen dilakukan pada empat skenario peta dengan lima seed acak. Hasil menunjukkan bahwa A* mempertahankan biaya optimal seperti UCS dengan ekspansi simpul lebih sedikit, GBFS paling cepat tetapi tidak selalu optimal, Brute Force hanya efektif pada peta sederhana karena ruang enumerasi cepat membesar, dan RRT* relevan untuk ruang kontinu meskipun performanya lebih bergantung pada sampling.

Keywords—*Brute Force, UCS, Greedy Best First Search, A*, RRT*, pencarian jalur, robot humanoid, analisis algoritma*

I. PENDAHULUAN

Robot sepak bola humanoid membutuhkan kemampuan memilih lintasan secara cepat karena lingkungan pertandingan dipenuhi objek yang dapat menghalangi gerak. Dalam tahap perencanaan gerak, robot perlu menentukan rute dari posisi awal menuju target sambil menghindari robot lain, batas area, atau zona yang tidak boleh dilalui.

Pencarian jalur relevan untuk Analisis dan Strategi Algoritma karena satu masalah dapat diselesaikan dengan pendekatan yang sangat berbeda. Pada makalah ini, algoritma yang dibandingkan disusun menjadi tiga pilihan utama: Brute Force, kelompok UCS/GBFS/A*, dan RRT*. Pengelompokan UCS, GBFS, dan A* mengikuti butir tugas yang menempatkan ketiganya sebagai satu strategi pencarian berbasis frontier.

Tujuan makalah ini adalah menerapkan dan membandingkan kelima algoritma tersebut pada peta simulasi yang sama. Kontribusi utama makalah adalah eksperimen komputasional yang memperlihatkan trade-off antara kelengkapan, biaya lintasan, waktu

eksekusi, dan jumlah simpul atau kandidat yang diproses.

II. BATASAN MASALAH

Penelitian dibatasi pada simulasi dua dimensi. Lapangan direpresentasikan sebagai bidang berukuran 60 x 40 satuan, sedangkan posisi awal dan posisi target dianggap sudah diketahui. Hambatan dimodelkan sebagai lingkaran dan persegi panjang yang tidak boleh dilewati lintasan.

Brute Force menggunakan enumerasi kandidat waypoint dan pemeriksaan garis bebas hambatan. UCS, GBFS, dan A* menggunakan grid delapan arah, yaitu atas, bawah, kiri, kanan, dan empat arah diagonal. Biaya gerak lurus bernilai 1, sedangkan biaya gerak diagonal bernilai akar dua. Pada RRT*, posisi robot direpresentasikan sebagai titik kontinu.

Aspek dinamika robot seperti keseimbangan, percepatan servo, gaya gesek kaki, sensor kamera, dan ketidakpastian lokalisasi tidak dimodelkan secara detail. Oleh karena itu, eksperimen ini lebih tepat disebut sebagai simulasi perencanaan jalur, bukan simulasi robot humanoid lengkap.

Secara formal, masukan masalah adalah ukuran lapangan, posisi awal robot, posisi tujuan, dan himpunan hambatan. Hambatan direpresentasikan sebagai lingkaran atau persegi panjang, sedangkan posisi start dan goal berada pada ruang bebas. Pada model grid, ruang dibagi menjadi sel diskret; pada model kontinu, posisi dianggap sebagai koordinat real yang dapat dihubungkan oleh segmen garis.

Keluaran yang diharapkan adalah lintasan valid dari start ke goal. Lintasan valid tidak boleh berpotongan dengan hambatan dan harus tetap berada di dalam batas lapangan. Fungsi objektif yang digunakan adalah meminimalkan biaya lintasan, yaitu jumlah jarak antar titik atau simpul berurutan. Dengan formulasi ini, perbedaan algoritma dapat dibandingkan melalui keberhasilan menemukan jalur, biaya rute, waktu komputasi, dan jumlah processed.

III. DASAR TEORI

A. Pemodelan Pencarian Jalur

Masalah pencarian jalur dapat dimodelkan sebagai pencarian lintasan dari titik awal ke titik tujuan pada ruang bebas hambatan. Dalam representasi grid, setiap sel bebas menjadi simpul dan perpindahan antar sel menjadi sisi. Dalam representasi waypoint atau ruang kontinu, simpul dapat berupa titik kandidat yang dihubungkan jika segmen garis antar titik tidak menabrak hambatan.

$$C(P) = \sum d(v_i, v_{i+1}), 0 \leq i < k \quad (1)$$

Persamaan (1) menunjukkan bahwa biaya lintasan merupakan penjumlahan jarak antar simpul atau titik berurutan. Untuk robot humanoid, biaya ini dapat dimaknai sebagai pendekatan terhadap jarak tempuh yang harus dilalui sebelum modul kontrol gerak menjalankan langkah fisik.

B. Brute Force Berbasis Waypoint

Brute Force mencoba semua kombinasi kandidat solusi dalam batas tertentu. Pada eksperimen ini, kandidat waypoint diambil dari koridor start-goal, titik sekitar hambatan, dan area celah dinding. Program kemudian mengevaluasi setiap urutan hingga tiga waypoint dan memilih rute bebas hambatan dengan biaya terkecil. Pendekatan ini mudah dipahami, tetapi tidak lengkap jika solusi membutuhkan waypoint di luar kandidat atau lebih banyak titik antara.

C. Kelompok UCS, Greedy Best First Search, dan A*

UCS, GBFS, dan A* sama-sama menggunakan frontier berbasis priority queue, tetapi berbeda pada fungsi prioritasnya. UCS memilih simpul dengan biaya aktual terkecil dari start. GBFS memilih simpul yang secara heuristik paling dekat ke tujuan. A* menggabungkan biaya aktual dan estimasi sisa biaya sehingga dapat lebih terarah tanpa kehilangan optimalitas ketika heuristiknya admissible [1], [2].

$$p_{UCS}(n) = g(n), \quad p_{GBFS}(n) = h(n), \quad p_{A^*}(n) = g(n) + h(n) \quad (2)$$

Heuristik yang digunakan adalah jarak Euclidean karena gerakan grid mengizinkan arah diagonal. UCS digunakan sebagai baseline optimal pada graf berbobot non-negatif, GBFS digunakan untuk melihat dampak keputusan yang sangat agresif mengikuti arah goal, dan A* digunakan sebagai kompromi antara optimalitas dan efisiensi.

D. Rapidly-exploring Random Tree Star

RRT* adalah pengembangan dari Rapidly-exploring Random Tree yang menumbuhkan pohon pencarian melalui sampling acak pada ruang konfigurasi. Setiap sampel dihubungkan ke simpul terdekat jika lintasan bebas hambatan, kemudian

dilakukan rewiring agar struktur pohon mendekati lintasan berbiaya lebih kecil [3], [4].

E. Metrik Evaluasi

Empat metrik digunakan dalam eksperimen. Pertama, tingkat keberhasilan menunjukkan persentase percobaan yang berhasil menemukan jalur. Kedua, biaya lintasan menunjukkan panjang jalur yang dihasilkan. Ketiga, waktu eksekusi mengukur durasi komputasi dalam milidetik. Keempat, jumlah processed menunjukkan simpul grid, node sampling, atau kandidat waypoint yang diproses.

F. Kompleksitas dan Karakter Implementasi

Brute Force memiliki kompleksitas yang dipengaruhi jumlah kandidat m dan batas kedalaman d , yaitu mendekati jumlah permutasi $\Sigma P(m, d)$. Pada eksperimen ini d dibatasi menjadi tiga agar masih dapat diuji. UCS, GBFS, dan A* pada priority queue memiliki kompleksitas teoretis mendekati $O((V + E) \log V)$, dengan V sebagai jumlah sel bebas dan E sebagai jumlah hubungan antar sel.

RRT* memiliki karakter berbeda karena biaya komputasi dipengaruhi jumlah sampel dan proses pencarian tetangga untuk rewiring. Implementasi sederhana yang digunakan masih melakukan pencarian tetangga secara linear, sehingga biaya dapat mendekati $O(n^2)$ terhadap jumlah node sampling.

TABLE I. RINGKASAN KOMPLEKSITAS LIMA ALGORITMA

Algoritma	Prioritas	Kompleksitas	Catatan
bruteforce	waypoint	$O(\Sigma P(m, d) \cdot L)$	Batas $d=3$
UCS	$g(n)$	$O((V + E) \log V)$	Optimal
GBFS	$h(n)$	$O((V + E) \log V)$	Tidak optimal
A*	$g(n) + h(n)$	$O((V + E) \log V)$	Optimal*
RRT*	Sampling	$O(n)^2$	Kontinu

IV. METODOLOGI

A. Desain Simulasi

Eksperimen dibuat menggunakan program Python. Peta simulasi berukuran 60 x 40 satuan dengan titik awal di sisi kiri lapangan dan titik tujuan di sisi kanan lapangan. Setiap peta berisi hambatan berbentuk lingkaran dan persegi panjang. Empat tingkat skenario digunakan agar algoritma diuji pada kondisi mudah hingga padat.

B. Lingkungan Eksperimen

Eksperimen dijalankan pada satu perangkat yang sama agar perbandingan waktu lebih konsisten. Program dijalankan melalui uv dengan CPython 3.12.13 pada Windows 11 dan memori sekitar 16 GB. Pengukuran waktu menggunakan `time.perf_counter` sehingga nilai yang dicatat adalah durasi eksekusi fungsi algoritma dalam milidetik.

Resolusi grid ditetapkan 60 x 40 sel, dengan lima seed acak untuk setiap skenario. Hasil setiap seed

disimpan dalam hasil_eksperimen.csv, sedangkan rata-rata biaya, waktu, tingkat keberhasilan, dan processed disimpan dalam ringkasan_eksperimen.csv. Seluruh visualisasi pada makalah dibuat ulang dari keluaran program yang sama.

TABLE II. SKENARIO PENGUJIAN

Algoritma	Hambatan	Tujuan uji
Mudah	5 lingkaran	Hambatan jarang
Sedang	10 lingkaran + 2 dinding	Ada celah lintasan
Sulit	2 dinding + 8 lingkaran	Perlu belokan jauh
Padat	19 lingkaran + 2 dinding	Ruang bebas sempit

C. Langkah Eksperimen

Setiap skenario diuji sebanyak lima kali dengan seed acak berbeda. Prosedur yang dilakukan adalah membangkitkan peta, menjalankan Brute Force, UCS, GBFS, A*, dan RRT*, lalu menyimpan biaya lintasan, waktu eksekusi, jumlah processed, dan status keberhasilan.

Pada Brute Force, program menyusun kandidat waypoint, memeriksa visibilitas antar titik, lalu mengevaluasi semua urutan kandidat sampai batas tiga waypoint. Pada UCS, GBFS, dan A*, frontier diimplementasikan menggunakan priority queue dengan fungsi prioritas masing-masing. Pada RRT*, parameter utama yang digunakan adalah jumlah iterasi maksimum, step size, radius rewiring, dan peluang sampling langsung ke goal.

D. Parameter Algoritma

TABLE III. PARAMETER ALGORITMA

Algoritma	Parameter
bruteforce	waypoints = 3; kandidat = 24 kandidat maks. 24, kedalaman waypoint maks. 3
UCS	$p(n) = g(n)$; grid 8 arah grid 60x40, 8 arah, biaya lurus 1 dan diagonal $\sqrt{2}$
GBFS	$p(n) = h(n)$; Euclidean grid 60x40, $h(n)$ =jarak Euclidean ke goal
A*	$p(n) = g(n) + h(n)$; Euclidean grid 60x40, $f(n)=g(n)+h(n)$, $h(n)$ =Euclidean
RRT*	iter = 650/950; step = 2,5; radius = 4,5max_iter 650 (Padat 950), step 2,5, radius 4,5, goal bias 0,12

Parameter Brute Force dibatasi agar eksperimen tetap selesai dalam waktu wajar. Parameter RRT* dipilih melalui percobaan awal agar algoritma memiliki peluang menemukan jalur pada skenario sulit, tetapi tetap dapat selesai dalam waktu yang sebanding dengan metode lain. Pada skenario padat, jumlah iterasi maksimum RRT* dinaikkan karena ruang bebas lebih sempit.

E. Rancangan Pseudocode Ringkas

Secara umum, Brute Force membangun daftar kandidat titik, lalu mencoba rute start-waypoint-goal secara menyeluruh sampai batas kedalaman. UCS,

GBFS, dan A* menggunakan kerangka program yang sama: memasukkan simpul awal ke priority queue, mengambil simpul dengan prioritas terkecil, memeriksa goal, lalu memasukkan tetangga valid dengan prioritas sesuai algoritma.

Untuk RRT*, program mengulang proses sampling titik acak, mencari simpul terdekat pada pohon, membuat titik baru sejauh step size, memeriksa tabrakan, memilih parent terbaik dari tetangga dalam radius tertentu, lalu melakukan rewiring jika jalur melalui titik baru menghasilkan biaya lebih rendah.

V. HASIL DAN PEMBAHASAN

A. Hasil Kuantitatif

Tabel IV menunjukkan ringkasan hasil eksperimen dari empat skenario. Nilai biaya, waktu, dan processed merupakan rata-rata dari lima percobaan. Jika suatu algoritma gagal pada sebagian seed, biaya rata-rata dihitung hanya dari percobaan yang berhasil sehingga tingkat keberhasilan harus dibaca bersama nilai biaya.

TABLE IV. RINGKASAN BIAYA DAN WAKTU EKSPERIMEN

Sken.	Algoritma	Ok	Biaya	Ms
Mudah	Brute Force	100%	53.34	118.8
Mudah	UCS	100%	54.82	240.2
Mudah	GBFS	100%	55.44	7.4
Mudah	A*	100%	54.82	31.4
Mudah	RRT*	100%	57.25	197.9
Sedang	Brute Force	80%	53.16	128.8
Sedang	UCS	100%	54.49	413.5
Sedang	GBFS	100%	55.44	11.1
Sedang	A*	100%	54.49	46.9
Sedang	RRT*	100%	56.34	288.8
Sulit	Brute Force	20%	66.13	94.6
Sulit	UCS	100%	70.76	342.2
Sulit	GBFS	100%	75.88	33.8
Sulit	A*	100%	70.76	241.1
Sulit	RRT*	80%	84.69	125.4
Padat	Brute Force	20%	86.53	91.9
Padat	UCS	80%	57.56	732.6
Padat	GBFS	80%	60.15	182.6
Padat	A*	80%	57.56	244.9
Padat	RRT*	80%	59.49	686.2

Selain biaya dan waktu, jumlah processed penting karena menunjukkan seberapa banyak kandidat solusi, simpul grid, atau node sampling yang harus diperiksa oleh algoritma. Karena satuan processed berbeda antarrepresentasi, angka pada Tabel V dibaca sebagai indikator beban kerja internal setiap algoritma, bukan sebagai satuan fisik yang sepenuhnya identik.

TABLE V. RINGKASAN JUMLAH PROCESSED

Sken.	Algoritma	Processed
Mudah	Brute Force	12721

Sken.	Algoritma	Processed
Mudah	UCS	2041
Mudah	GBFS	54.6
Mudah	A*	246.2
Mudah	RRT*	617
Sedang	Brute Force	12721
Sedang	UCS	1828.2
Sedang	GBFS	54.8
Sedang	A*	226.6
Sedang	RRT*	576.2
Sulit	Brute Force	12721
Sulit	UCS	1808
Sulit	GBFS	140.2
Sulit	A*	1037
Sulit	RRT*	372
Padat	Brute Force	12721
Padat	UCS	1610.6
Padat	GBFS	259.2
Padat	A*	518.2
Padat	RRT*	690

UCS memproses simpul lebih banyak karena eksplorasinya melebar berdasarkan biaya aktual. A* menekan jumlah processed dengan memanfaatkan heuristik Euclidean, sedangkan GBFS paling sedikit memproses simpul tetapi tidak menjamin biaya minimum. Brute Force selalu mengevaluasi 12.721 kandidat rute pada konfigurasi batas yang digunakan, sementara processed RRT* bergantung pada jumlah node sampling yang berhasil ditambahkan.

Pada skenario mudah, seluruh algoritma berhasil menemukan jalur. GBFS menjadi metode grid tercepat dengan waktu rata-rata 7,4 ms, tetapi biaya lintasannya 55,44, lebih tinggi daripada UCS dan A* yang sama-sama 54,82. Brute Force menghasilkan biaya 53,34 karena memakai segmen waypoint kontinu, tetapi tetap memeriksa 12.721 kandidat rute. RRT* berhasil menemukan jalur dengan biaya 57,25 dan waktu 197,9 ms.

Pada skenario sedang, Brute Force hanya berhasil pada 80% percobaan karena kandidat waypoint tidak selalu cukup untuk melewati celah hambatan. UCS, GBFS, A*, dan RRT* berhasil 100%. A* mempertahankan biaya optimal seperti UCS, yaitu 54,49, tetapi waktunya lebih kecil, sekitar 46,9 ms dibanding 413,5 ms pada UCS. GBFS tetap menjadi metode grid tercepat pada 11,1 ms, sedangkan RRT* membutuhkan 288,8 ms.

Pada skenario sulit, Brute Force turun menjadi 20% keberhasilan. UCS, GBFS, dan A* tetap berhasil 100%, sedangkan RRT* berhasil 80%. GBFS menjadi

metode grid tercepat pada 33,8 ms, tetapi biaya rata-ratanya 75,88, lebih buruk daripada UCS dan A* yang berada pada 70,76. Hal ini menunjukkan konsekuensi pemilihan heuristik tanpa memperhitungkan biaya aktual secara penuh.

Pada skenario padat, semua metode menghadapi kondisi paling berat. Brute Force hanya berhasil 20%, sedangkan UCS, GBFS, A*, dan RRT* berhasil 80%. A* kembali mempertahankan biaya sama dengan UCS, yaitu 57,56, sementara GBFS menghasilkan biaya 60,15 dan RRT* 59,49. Waktu UCS dan RRT* meningkat menjadi 732,6 ms dan 686,2 ms karena eksplorasi, sampling, dan rewiring lebih sering terhambat ruang bebas yang sempit.

Secara umum, hasil menunjukkan bahwa Brute Force cocok sebagai baseline konseptual tetapi tidak kuat untuk peta kompleks. GBFS unggul dari sisi waktu dan jumlah processed, tetapi kualitas jalurnya tidak selalu baik. A* menjadi metode grid paling seimbang karena mempertahankan optimalitas UCS dengan ekspansi lebih sedikit. RRT* relevan untuk ruang kontinu, tetapi performanya lebih sensitif terhadap sampling dan parameter.

Visualisasi Jalur

Gambar 1 memperlihatkan contoh lintasan pada skenario sulit seed 4, yaitu seed yang menghasilkan lintasan valid untuk semua metode. Brute Force menghasilkan lintasan berupa segmen waypoint, UCS dan A* mengikuti grid biaya minimum, GBFS mengambil rute yang lebih agresif menuju goal, dan RRT* membentuk lintasan kontinu dari pohon sampling.

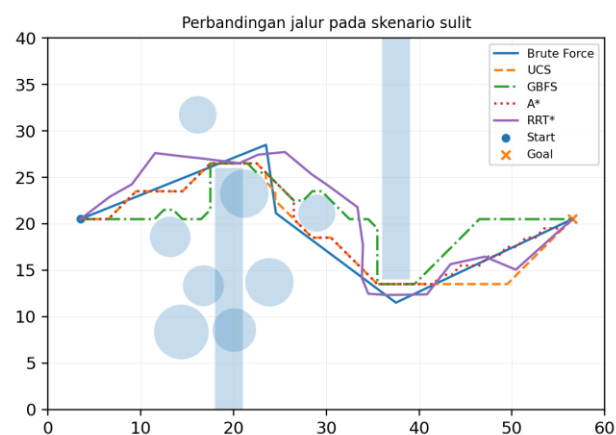


Fig. 1. Perbandingan lintasan Brute Force, UCS, GBFS, A*, dan RRT* pada skenario sulit.

Gambar 2 menunjukkan perbandingan waktu eksekusi rata-rata, dengan batang galat yang menyatakan simpangan baku sampel dari lima seed. GBFS menjadi metode grid tercepat pada setiap skenario, sedangkan A* mempertahankan biaya UCS dengan waktu lebih rendah. Variasi waktu membesar

pada skenario padat, terutama pada GBFS, A*, dan RRT*, sehingga hasil waktu perlu dibaca bersama variasi antarseed.

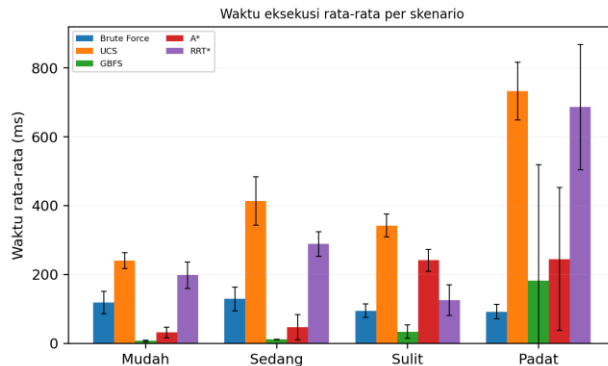


Fig. 2. Waktu eksekusi rata-rata dan simpangan baku pada setiap skenario.

Gambar 3 membandingkan jumlah processed pada keluarga pencarian grid, sehingga satuannya konsisten sebagai simpul grid yang diproses. Dibandingkan UCS, A* mengurangi processed sebesar 42,6-87,9%: Mudah 87,9%, Sedang 87,6%, Sulit 42,6%, dan Padat 67,8%. Pada keempat skenario, pengurangan ini tetap menghasilkan biaya lintasan dan tingkat keberhasilan yang sama dengan UCS.

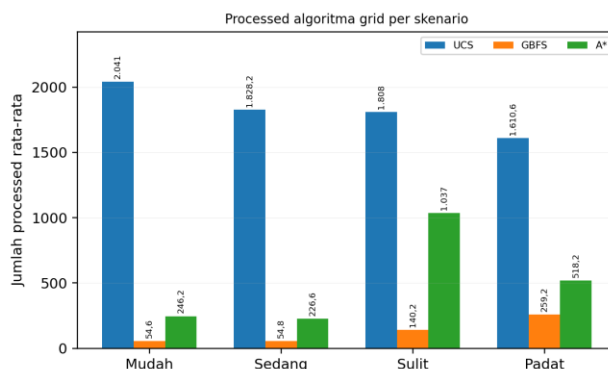


Fig. 3. Jumlah processed rata-rata UCS, GBFS, dan A* pada setiap skenario.

B. Analisis Perbandingan Algoritma

Brute Force memiliki nilai pedagogis karena memperlihatkan cara paling langsung untuk mencoba kandidat solusi. Namun, pendekatan ini cepat kehilangan kelengkapan ketika jumlah kandidat dibatasi dan cepat kehilangan efisiensi ketika batasnya diperbesar. Dalam konteks robot sepak bola, Brute Force lebih cocok sebagai baseline kecil, bukan strategi utama.

Kelompok UCS, GBFS, dan A* memperlihatkan spektrum strategi frontier. UCS paling aman untuk memverifikasi biaya optimal, tetapi cenderung banyak mengekskansi simpul. GBFS sangat cepat karena langsung mengejar goal, tetapi dapat memilih lintasan

yang lebih panjang. A* menempati posisi tengah yang paling praktis karena heuristik membantu mengurangi ekspansi tanpa mengorbankan optimalitas pada grid ini.

RRT* memiliki keunggulan konseptual karena tidak harus membagi ruang menjadi grid. Pendekatan ini relevan untuk perencanaan gerak kontinu dan dapat dikembangkan untuk mempertimbangkan batasan robotik lebih nyata, termasuk melalui varian informed sampling seperti Informed RRT* [7]. Kekurangannya adalah performa lebih sulit diprediksi karena bergantung pada jumlah sampel, goal bias, dan konfigurasi hambatan.

C. Implikasi untuk Robot Sepak Bola Humanoid

Jika sistem robot membutuhkan keputusan cepat pada peta grid lokal, A* merupakan pilihan paling kuat karena kompromi antara kualitas solusi dan efisiensi komputasi. GBFS dapat digunakan ketika waktu sangat sempit dan sedikit penurunan kualitas jalur masih dapat diterima. UCS berguna sebagai pembanding optimal, Brute Force sebagai baseline, dan RRT* untuk pengembangan menuju ruang gerak kontinu.

D. Validasi dan Keterbatasan Eksperimen

Validasi dilakukan dengan memeriksa bahwa jalur tidak memotong area hambatan. Pada Brute Force dan RRT*, setiap segmen garis diuji terhadap hambatan. Pada UCS, GBFS, dan A*, validasi dilakukan melalui pemeriksaan sel tetangga dan garis antar pusat sel agar gerak diagonal tidak melewati sudut hambatan.

Keterbatasan utama eksperimen adalah bentuk robot direduksi menjadi titik. Dalam robot humanoid nyata, tubuh robot memiliki lebar, postur, dan stabilitas berjalan yang perlu diperhitungkan. Selain itu, peta bersifat statis, sedangkan pertandingan robot sepak bola memiliki objek dinamis dan ketidakpastian sensor.

Sebagai bagian dari validasi, perbandingan ini tidak sepenuhnya identik karena setiap pendekatan memakai representasi ruang yang berbeda. UCS, GBFS, dan A* bekerja pada grid diskret; Brute Force bekerja pada kandidat waypoint dan segmen garis; sedangkan RRT* bekerja melalui sampling di ruang kontinu. Akibatnya, biaya lintasan A* dapat disebut optimal pada representasi grid, tetapi tidak otomatis menjadi optimum global pada ruang kontinu.

Ancaman validitas lain berasal dari jumlah seed yang terbatas, implementasi Python yang belum dioptimasi tingkat rendah, serta parameter Brute Force dan RRT* yang perlu dibatasi agar eksperimen selesai dalam waktu wajar. Tabel utama menampilkan rata-rata agar tetap ringkas, sedangkan data mentah per seed dan simpangan baku disimpan pada CSV hasil eksperimen. Pada skenario Padat, simpangan baku waktu GBFS, A*, dan RRT* masing-masing sekitar 336,8 ms, 207,6 ms, dan 181,6 ms; simpangan baku

processed RRT* sekitar 206,8 node. Variasi ini menunjukkan bahwa pengukuran waktu dan metode sampling perlu dibaca bersama sebaran antarseed. Untuk menjaga fairness, seluruh algoritma dijalankan pada peta yang sama, menggunakan perangkat yang sama, dan dievaluasi dengan fungsi validasi tabrakan yang sama untuk memastikan jalur tidak memotong hambatan.

VI. KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan eksperimen yang dilakukan, A* menjadi algoritma paling seimbang untuk simulasi pencarian jalur robot sepak bola humanoid berbasis grid. A* menghasilkan biaya lintasan yang sama dengan UCS pada seluruh skenario yang berhasil, tetapi dengan waktu eksekusi dan jumlah ekspansi simpul yang lebih rendah.

GBFS terbukti sangat cepat, tetapi tidak menjamin biaya minimum. Brute Force memperlihatkan batas pendekatan enumeratif: performanya masih dapat diterima pada peta sederhana, tetapi tingkat keberhasilannya turun pada skenario sulit dan padat. RRT* menawarkan fleksibilitas ruang kontinu, namun membutuhkan tuning sampling agar stabil.

Dengan demikian, untuk kasus lapangan berhalangan yang masih berbasis grid, A* lebih direkomendasikan sebagai strategi utama. Untuk penelitian robotika lanjutan yang mempertimbangkan ruang gerak kontinu dan batasan kinematika, RRT* tetap relevan sebagai dasar pengembangan.

B. Saran

Penelitian selanjutnya dapat menambahkan hambatan dinamis agar lebih mendekati kondisi pertandingan robot sepak bola. Eksperimen juga dapat diperluas dengan variasi algoritma seperti Theta* [8], D* Lite [9], atau Hybrid A* [10] yang lebih dekat dengan kebutuhan navigasi robot bergerak. Pada sisi Brute Force, kandidat waypoint dapat dibuat adaptif menggunakan visibility graph agar baseline lebih kuat. Pada sisi RRT*, tuning parameter dapat dilakukan secara lebih sistematis dengan menguji variasi step size, radius rewiring, goal bias, jumlah iterasi, dan informed sampling [7].

LAMPIRAN

Link kode program:
<https://github.com/myudak/Project-ASA-Analisis-Perbandingan-Path.git>.

Link video penjelasan:
<https://github.com/myudak/Project-ASA-Analisis-Perbandingan-Path.git>.

REFERENCES

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [4] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [5] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Cham, Switzerland: Springer, 2017.
- [6] M. Bennewitz, S. Behnke, and W. Burgard, "Optimizing humanoid walking paths using body motion primitives," in *Proc. IEEE International Conference on Robotics and Automation*, 2005, pp. 1273–1278.
- [7] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 2997–3004.
- [8] A. Nash, K. Daniel, S. Koenig, and A. Felner, "Theta*: Any-angle path planning on grids," *Journal of Artificial Intelligence Research*, vol. 39, pp. 533–579, 2010.
- [9] S. Koenig and M. Likhachev, "D* Lite," in *Proc. AAAI Conference on Artificial Intelligence*, 2002, pp. 476–483.
- [10] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Practical search techniques in path planning for autonomous driving," in *Proc. AAAI Workshop on Search in Artificial Intelligence and Robotics*, 2008.

PERNYATAAN

Dengan ini saya menyatakan bahwa makalah yang saya susun merupakan hasil karya saya sendiri dan tidak merupakan saduran, terjemahan, maupun plagiasi dari karya orang lain. Seluruh sumber yang digunakan telah dicantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Terkait penggunaan teknologi kecerdasan buatan (Artificial Intelligence), saya menyatakan bahwa:

☐ Saya tidak menggunakan alat bantu AI dalam proses penyusunan makalah ini.

☒ Saya menggunakan alat bantu AI sebagai pendukung dalam penyusunan makalah ini dengan rincian sebagai berikut:

- Nama alat AI: OpenAI Codex.
- Tujuan penggunaan: membantu revisi struktur algoritma, perapian narasi, dan penyelarasan hasil eksperimen dengan kode.
- Ruang lingkup penggunaan: penyusunan teks makalah, bantuan pembaruan kode eksperimen, dan perapian tabel/visualisasi; seluruh isi, analisis, dan keputusan akhir tetap diverifikasi.

Penggunaan AI, apabila ada, hanya bersifat sebagai alat bantu dan tidak menggantikan peran saya sebagai penulis utama. Seluruh isi, analisis, serta kesimpulan dalam makalah ini tetap merupakan tanggung jawab saya sepenuhnya.

Semarang, 9 Mei 2026



Muchammad Yuda Tri Ananda
24060124110142

